

```

from abc import ABC, abstractmethod

# =====
# MIXIN
# =====
class LoginMixin:
    def login(self):
        print("Login berhasil")

    def logout(self):
        print("Logout berhasil")

class NotificationMixin:
    def kirim_notifikasi(self, pesan):
        print(f"Notifikasi: {pesan}")

# =====
# STATE MACHINE
# =====
class StateMachine:
    def __init__(self):
        self.state = "Aktif"

    def change_state(self, state_baru):
        self.state = state_baru
        print(f"State berubah menjadi: {self.state}")

# =====
# ABSTRACT CLASS
# =====
class Mahasiswa(ABC, LoginMixin, NotificationMixin):

    def __init__(self, nama, nim):
        # ENKAPSULASI
        self.__nama = nama
        self.__nim = nim

    # Getter
    def get_nama(self):
        return self.__nama

```

```

def get_nim(self):
    return self.__nim

# Setter
def set_nama(self, nama_baru):
    self.__nama = nama_baru

# ABSTRACT METHOD
@abstractmethod
def tampilkan_info(self):
    pass

# =====
# INHERITANCE
# =====
class MahasiswaInformatika(Mahasiswa, StateMachine):

    def __init__(self, nama, nim, bahasa):
        Mahasiswa.__init__(self, nama, nim)
        StateMachine.__init__(self)

        self.bahasa = bahasa

# POLYMORPHISM
def tampilkan_info(self):
    print("==== MAHASISWA INFORMATIKA =====")
    print("Nama      :", self.get_nama())
    print("NIM       :", self.get_nim())
    print("Bahasa    :", self.bahasa)
    print("Status   :", self.state)
    print()

class MahasiswaSistemInformasi(Mahasiswa, StateMachine):

    def __init__(self, nama, nim, bidang):
        Mahasiswa.__init__(self, nama, nim)
        StateMachine.__init__(self)

        self.bidang = bidang

# POLYMORPHISM

```

```

def tampilkan_info(self):
    print("===== MAHASISWA SISTEM INFORMASI =====")
    print("Nama      :", self.get_nama())
    print("NIM       :", self.get_nim())
    print("Bidang    :", self.bidang)
    print("Status   :", self.state)
    print()

# =====
# CLASS DOSEN
# =====

class Dosen:
    def __init__(self, nama, nidn):
        self.nama = nama
        self.nidn = nidn

    def tampilkan_dosen(self):
        print("Nama Dosen :", self.nama)
        print("NIDN      :", self.nidn)
        print()

# =====
# CLASS MATA KULIAH
# =====

class MataKuliah:
    def __init__(self, kode, nama, sks):
        self.kode = kode
        self.nama = nama
        self.sks = sks

    def tampilkan_matkul(self):
        print("Kode MK :", self.kode)
        print("Nama MK :", self.nama)
        print("SKS     :", self.sks)
        print()

# =====
# CLASS NILAI
# =====

class Nilai:
    def __init__(self, tugas, uts, uas):

```

```

        self.tugas = tugas
        self.uts = uts
        self.uas = uas

    def hitung_nilai(self):
        total = (self.tugas + self.uts + self.uas) / 3
        return total

# =====
# CLASS KELAS
# =====
class Kelas:
    def __init__(self, nama_kelas):
        self.nama_kelas = nama_kelas
        self.daftar_mahasiswa = []

    # CRUD TAMBAH
    def tambah_mahasiswa(self, mahasiswa):
        self.daftar_mahasiswa.append(mahasiswa)
        print(f"{mahasiswa.get_nama()} berhasil ditambahkan")

    # CRUD HAPUS
    def hapus_mahasiswa(self, mahasiswa):
        self.daftar_mahasiswa.remove(mahasiswa)
        print(f"{mahasiswa.get_nama()} berhasil dihapus")

    def tampilkan_semua(self):
        print(f"\n===== DAFTAR MAHASISWA {self.nama_kelas} =====")
        for mhs in self.daftar_mahasiswa:
            mhs.tampilkan_info()

# =====
# CLASS SISTEM KAMPUS
# =====
class SistemKampus:
    def __init__(self):
        self.daftar_dosen = []
        self.daftar_matkul = []

    def tambah_dosen(self, dosen):
        self.daftar_dosen.append(dosen)

```

```
def tambah_matkul(self, matkul):
    self.daftar_matkul.append(matkul)

# =====
# OBJECT
# =====

# Mahasiswa
mhs1 = MahasiswaInformatika(
    "tejo",
    "1234567890",
    "Python"
)

mhs2 = MahasiswaSistemInformasi(
    "acop",
    "231002",
    "Database"
)

# Dosen
dsn1 = Dosen(
    "Rafa",
    "112233"
)

# Mata Kuliah
mk1 = MataKuliah(
    "PB0101",
    "Pemrograman berorientasi objek ",
    3
)

# Nilai
nilai1 = Nilai(
    80,
    85,
    90
)
```

```
# Kelas
kelasA = Kelas("A")

# =====
# MENJALANKAN PROGRAM
# =====

# Login
mhs1.login()

# Tambah mahasiswa
kelasA.tambah_mahasiswa(mhs1)
kelasA.tambah_mahasiswa(mhs2)

# Tampilkan semua mahasiswa
kelasA.tampilkan_semua()

# State Machine
mhs1.change_state("Cuti")

# Kirim notifikasi
mhs1.kirim_notifikasi("Jadwal kuliah dimulai")

# Tampilkan dosen
dsn1.tampilkan_dosen()

# Tampilkan mata kuliah
mkl.tampilkan_matkul()

# Hitung nilai
print("Nilai Akhir :", nilai1.hitung_nilai())
```